

Pi Stuff Dialog 可读性重做报告

设计决策、具体示例与浅色预览。用于检查 8 个现有 Command Dialog 是否真的更容易读，而不是只看起来更统一。

Pi Stuff

Review draft · 2026-08-17

已接受的设计决策 · UI 实现仍待完成

| 目录

01	执行摘要	3
02	范围与共同语言	4
03	/agents	6
04	/tools 与 /tasks	8
05	/ctx、/diagnostics 与 /btw	10
06	/mcp 与 /rtk	12
07	实现顺序与验收	13
08	附录	14

执行摘要

这次重做不是把所有 Dialog 画成同一张表，而是统一它们的阅读语言，再让每个 Dialog 按自己的任务展示最重要的信息。共同规则负责降低学习成本，各自的内容结构负责让用户在几秒内看懂当前页面。

先说结论

- 共有 8 个现有非 SettingsList Command Dialog 进入本次讨论，设计均已接受，代码实现仍待完成。
- 所有状态都使用有固定含义的图标，> 只表示当前选中项，不再同时承担状态含义。
- PageUp/PageDown 保留，同时增加 Shift+Up/Down，照顾没有独立翻页键的 60% 键盘。
- 所有详情页使用 ◆ 标出板块标题，板块正文不继续缩进，避免三重缩进。
- Conversation Transcript、SettingsList、Agent roster、Todo 和 Statusline 不在这次 Dialog 重做范围内。

报告标题：Pi Stuff Dialog 可读性重做报告。

这份报告里的标记	意思
已决定	设计已经写入 ADR 或模块 README，可作为实现依据。
待实现	当前源代码仍是旧页面，预览图不能当成已经上线的截图。
不在范围	本次不顺手重做 Transcript、SettingsList、Roster、Todo 或 Statusline。

怎样检查这份报告：先看每页顶部的“一句话判断”，再看等宽示例。生成图只帮助检查层级、密度和浅色视觉，精确文案与图标含义以等宽示例和表格为准。

| 范围与共同语言

先统一阅读语言，不统一内容模板

8 个 Dialog 应该让人一眼认出它们属于同一个产品，但不应该强迫它们展示同样的板块。共同语言只管选择、状态、板块、翻页和退出，各 Dialog 仍按自己的任务组织内容。

共同语言包含 5 件事：> 只表示选中，状态由固定图标表达，详情板块用 ◆ 标题，翻页同时支持 PageUp/PageDown 与 Shift+Up/Down，Esc 始终沿着详情、列表、Dialog 的层级返回。用户学会一次，就能在 8 个页面复用。

真正需要区别的是内容和阅读方式。/agents 围绕 Agent 名称、任务和活动来读，并始终保持单栏；/tools、/tasks、/diagnostics 需要反复对照列表与详情，可以在宽屏使用双栏；裸 /btw 只在浏览历史时使用双栏。/ctx、/mcp、/rtk 继续单栏。双栏不是所有列表页的默认模板。

本次需要检查的 8 个 Dialog

命令	用户打开它时最想知道什么	页面形状
/agents	哪个 Agent，正在做什么，完整活动与结果是什么	列表 + 多层详情
/tools	这一段工具工作做了什么，某次调用的详情是什么	Activity 列表 + 调用详情
/tasks	现在还有哪些后台工作，输出或监看证据是什么	当前工作列表 + 类型专用详情
/ctx	上下文用了多少，哪里需要注意，现在能做什么	状态概览 + 操作流程
/diagnostics	出了什么问题，我现在能做什么	问题列表 + 诊断详情
/btw	这个问题的答案是什么，历史答案在哪里	直接答案 + 成功历史
/mcp	每个服务器是否可用，是否需要认证或重连	静态状态页
/rtk	运行环境是否可信，哪些行为开启，本次节省多少	静态检查页

共同的状态与选择语言

“统一图标”不等于每个图标在所有页面都只有一个单词。它统一的是大类：空心圆常表示尚未进入稳定工作，实心圆表示正在进行，感叹号表示需要注意，勾表示成功，叉表示失败，方块表示停止或禁用。详情里仍保留完整状态词，避免用户只靠颜色或记图标猜意思。

页面	紧凑图标	完整状态词仍然出现在哪里
Agent		详情页 Header
Tool Activity		Activity 详情 Header
Diagnostics		详情页严重程度文字
MCP		每一行服务器状态词
RTK		Runtime 状态词与开关词

板块与缩进

Agent name · ● running · 36s

◆ Task

Review Dialog readability at 64 and 32 columns.

◆ Activity

✓ Read agent-dialog.ts · completed

└ 24 lines shown · 186 lines omitted

Shift+↑/↓ page · Esc back

菱形图标只在板块标题这一行出现，不表示状态、选择或 Transcript 事件。标题、正文、Agent 名称、工具行、└ 预览和换行后的内容共享同一个左边界。这样保留层级，但不会越套越深。

不在范围

Conversation Transcript 的小圆点 ● 已经确定，本次不拿 Dialog 的状态图标去改它。Pi 原生 SettingsList 也保留原生键盘行为。Agent roster、Todo、BTW、Permission 和 Tool 状态继续各守一个权威位置，不复制到 Statusline 或另一个常驻面板。

| /agents: 先认出是谁, 再看它做了什么

Agent 名称是 `/agents` 的第一阅读锚点。列表先展示名称, 详情页再按 Task、可选结果和 Activity 展开, 用户不需要先穿过 State、Task、Transcript 三行才能知道自己正在看谁。

列表保持启动顺序, 不按状态重新排序。每行从 Agent 名称开始, 后面可带一句任务摘要, 最右侧才是状态图标与时间。这样状态变化不会让整张列表跳动, 用户也能凭名称稳定地找到同一个 Agent。

详情页在任何状态下都使用同一副单栏骨架: 顶部是 Agent 名称和完整状态, Task 永远存在, Result、Error 或 Partial result 只在确实有内容时出现, Activity 永远存在。更宽的终端把空间留给当前 Agent 的内容, 不保留 roster 栏。Activity 保留完整事件顺序, 但每次工具输出只给有限预览, 并明确说明省略了多少行。

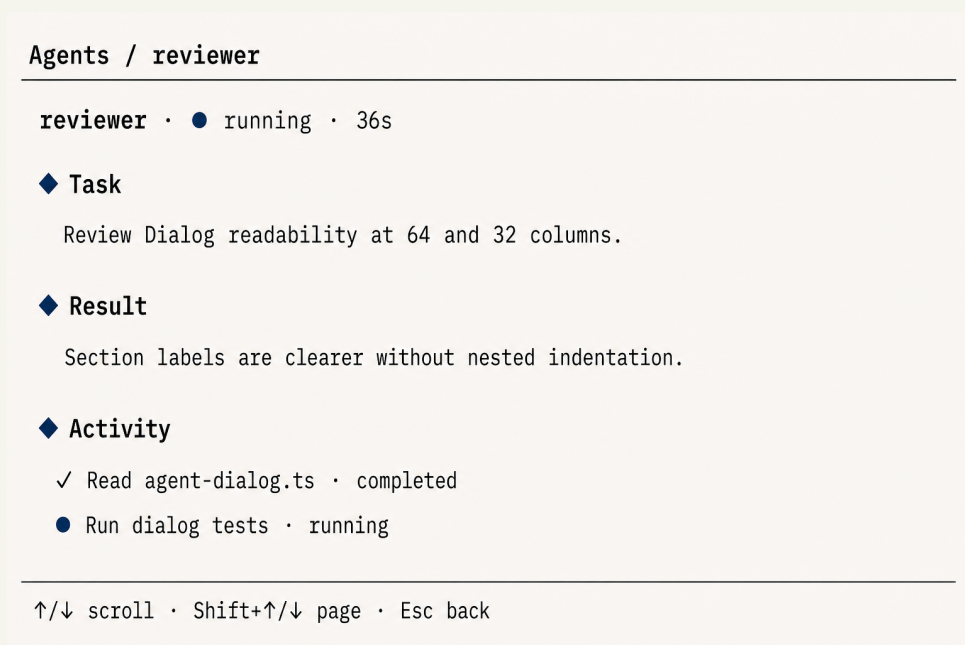


图 1: 浅色目标预览, 不是当前实现截图。Agent 详情始终使用单栏; 名称先建立身份, 菱形图标再把任务、结果和活动分成可扫描的板块。

具体详情示例

```
Agents / reviewer
✓ completed · 18s

◆ Task
Review the /agents dialog for readability at 64 and 32 columns.

◆ Result
The Agent name needs priority. Section labels should be stronger,
and Activity should keep every event while shortening large Tool output.

◆ Activity
reviewer
I'll inspect the current list and detail layout.

✓ Read agent-dialog.ts · completed
└ 24 lines shown · 186 lines omitted

↑/↓ scroll · Shift+↑/↓ page · r resume · Esc back
```

Activity 展示多少

这里的 Activity 指“按真实先后顺序发生的可见活动”。它不是只留三条最新记录, 也不是把整个原始协议文件原样倒出来。Agent 说过的话、用户明确给过的 steer 或 resume、每次 Tool 的目标、结果和有限预览都保留; 内部系统记录、原始 JSON、重复 Task 和重复最终 Result 不出现。

大输出只在自己的位置缩短, 例如写成“24 lines shown · 186 lines omitted”。页面不会因为输出大, 就悄悄删掉更早或更晚的 Activity。运行中的 Tool 也只占一行, 从 ● 原地更新为 ✓ 或 ×, 不会完成后再追加一份。

不同状态, 共用同一副骨架

Agent 状态	结果板块	Footer 中可用动作
queued / running	不出现	s steer、x stop
waiting	不出现	s reply、x stop
stopping	不出现	不显示会被拒绝的 stop
completed	Result	r resume
failed / crashed	Error, 可带 Partial result	r resume
stopped / cancelled	有内容时才出现 Partial result	stopped 可 resume, cancelled 无动作

04 · TOOL AND BACKGROUND WORK

| /tools 与 /tasks: 工作记录不能混在一起

/tools: 按一次工作来读, 不按协议记录来读

/tools 的主角是用户能理解的一次 Activity, 也就是一段连续的工具工作, 而不是内部调用 ID。列表先写它做了什么, 详情再按需要展开调用和格式化结果。

列表按最新 Activity 在前排列。每行依次是选择标记、可读摘要、状态图标和可选调用次数, 例如 Read · tool-dialog.ts ✓ 3 calls。屏幕变窄时先去掉调用次数, 不能先丢掉摘要或状态。

包含多次调用的 Activity 展示 Calls 和 Detail 两个板块, 单次调用直接进入 Detail, 不为一个条目造空列表。正常详情保留各工具自己的格式, Raw 只在调试时打开。宽屏双栏仍是一块固定高度的 Dialog, 只保留一条贯穿内容区的粗分隔线; 删除原型里容易被误读的标题短竖线, 改用栏标题强调色表示焦点。◆ 只表示板块, > 只表示选中行。

Tools · 4 activities	Read · tool-dialog.ts · ✓ completed · 3 calls
> Read · tool-dialog.ts ✓ 3 calls Search dialog contracts ✓ 8 calls Run focused tests ● 2 calls Apply patch ✕	◆ Calls > ✓ Read tool-dialog.ts ✓ Read README.md ✓ Read ADR 0010 ◆ Detail Target: packages/pi-stuff/src/tool-display/tool-dialog.ts L 214 lines
↑/↓ select · Shift+↑/↓ page · Enter open · Esc back	

图 2: 浅色目标预览。双栏共用一条贯穿式中线; 当前栏用标题强调色表示焦点。

```
Tools / Read 4 files and searched 2 patterns
✓ success · 6 calls

◆ Calls
> ✓ Read · packages/pi-stuff/src/tool-display/tool-dialog.ts
  ✓ Search · toolStateGlyph

◆ Detail · formatted
Read
Target: packages/pi-stuff/src/tool-display/tool-dialog.ts
Summary: 58 lines returned
```


Raw protocol 的意思是“调用时保存的原始协议数据”。它包含 call ID、Tool 名称、参数、结果和 details，只为调试服务。按 `r` 在 formatted 与 Raw 之间切换，Esc 先从 Raw 回 formatted，再从详情回列表。

/tasks: 只管当前仍在进行的后台工作

`/tasks` 是当前工作台，不是历史记录。它只展示仍在运行、等待、停止中或恢复中的 Shell、Monitor 和 Agent 投影，并把三种任务的详情分开设计。

列表先写任务类型和可辨认的名称，再写一句短说明，右侧放状态与已用时间。Agent 行仍以 Agent 名称为核心，Enter 直接进入 `/agents`；`/tasks` 不再复制一份较差的 Agent 详情。

Shell 详情需要 Command 与 Output，因为用户关心命令和输出。Monitor 详情需要 Source、Condition 与 Latest evidence，因为用户关心它在监看什么、何时结束、最新看到了什么。宽屏时，列表与当前 Shell 或 Monitor 详情并列，方便连续检查多个后台工作；Agent 行仍进入单栏 `/agents`。窄屏恢复顺序式列表和详情。



图 3: 这里只是并列比较 `/tasks` 与 `/ctx`，不是一个双栏 Dialog。真正的 `/tasks` 双栏由任务列表和当前 Shell 或 Monitor 详情组成。

任务类型	详情板块	谁拥有控制权
Shell	Command、Output	<code>/tasks</code> 可 stop
Monitor	Source、Condition、Latest evidence	<code>/tasks</code> 可 stop
Agent 投影	不在此复制详情	Enter 打开 <code>/agents</code>

两页的边界：`/tools` 看 Tool 调用和协议详情，`/tasks` 看当前后台工作。Tool 历史不能混进当前任务列表，已经结束后台工作也不在 `/tasks` 里长期堆积。

| /ctx、/diagnostics 与 /btw: 先回答眼前的问题

/ctx: 先告诉用户还剩多少空间

/ctx 的第一任务是回答上下文用了多少、是否需要处理，而不是先展示内部组件名。页面顺序改为 Usage、Overview、Attention、Actions，内部概念在出现时立刻用普通话解释。

Usage 放最前面，用实际占用和剩余空间建立判断。Compartment 可以解释成“上下文里的分区”，Historian 可以解释成“负责压缩和回顾旧内容的后台整理器”，pending drop 可以解释成“已标记、稍后会从上下文中移除的内容”。

不会产生任何变化的 flush 或 upgrade 行默认不显示，但命令仍然可用。重建这类高影响动作使用带警告图标的确认页，并默认选中 Cancel；确认后 Dialog 先关闭，进度继续写入原有 Context Activity，不再叠一个进度窗口。

```
Context
72.4% · 145K / 200K tokens

◆ Overview
8 compartments · compacted history ~54K tokens
Historian idle · cache 12m remaining

◆ Attention
! 2 pending drops · removals waiting to apply

◆ Actions
> Wrap up history
  Apply 2 pending drops
  Rebuild compartments
```

重建确认必须明确说后果

```
Context / Confirm rebuild

! Rebuild the full eligible session?
Compartment and fact data will be regenerated and may use model tokens.
The original Pi conversation is kept.

> Cancel
  Rebuild now
```

/diagnostics: 先看问题，再看是谁报的

诊断记录要先让用户看懂发生了什么，再告诉他来自哪个 Capability。严重程度图标只表达信息、警告和错误，不借用运行中的实心圆。

列表使用 `i`、`!`、`x`，按最新更新时间排序。每行依次展示来源、问题摘要、重复次数和时间；窄屏保留图标、来源和有意义的摘要。

详情用 Action 说明下一步，用 Details 放诊断上下文。宽屏保留左侧列表和右侧当前详情，窄屏顺序打开。Severity、Occurred 已由图标、排序和页头表达，不再重复占行。

Diagnostics · 3 records

<code>x</code> Agents	Failed to read Agent transcript	<code>x3</code> · 2m
<code>!</code> Background Work	Could not confirm process ownership	5m
<code>i</code> Context	Retried a stale derived-state refresh	now

Diagnostics / Agents

`x` error · 3 occurrences · latest 2m ago

Failed to read Agent transcript

◆ Action

Run `/agents` again after the session file becomes readable.

◆ Details

...latest bounded and redacted diagnostic detail...

/btw: 问完直接看答案，历史只是需要时再找

`/btw question` 应直接进入当前答案，不先让用户穿过历史列表。只有裸 `/btw` 才打开历史；流式答案跟随用户是否停在底部，不强行把滚动位置拉回去。

Answer 是主板块，状态图标区分生成、完成、取消和失败。向上滚动会暂停自动跟随并提示新内容，回到底部才恢复。

历史只保存成功交流，不重复成功图标。裸 `/btw` 在宽屏可并列历史与已完成答案；直接提问和流式答案始终单栏。Footer 写 `f new session`，清理历史仍需确认；等待中的回答不能被 Enter 或 Space 悄悄关闭。

BTW · 3 of 4

Why did the typecheck fail? · `✓` answered

◆ Answer

...Markdown answer...

`</>` history · `Shift+↑/↓` page · `c` copy · `f` new session · `Esc` close

| /mcp 与 /rtk: 静态页也要有清楚的主次

静态状态页不需要为了统一而增加列表详情层。/mcp 以服务器名称和可用性为主，/rtk 以运行环境、行为开关和本次节省为主，两者都在一页内完成阅读。

/mcp 每行先写服务器名称，再用 ✓、○、×、!、■ 表达状态并保留状态词。资源和工具数量放在后面，认证或重连说明必须在能力列表之前出现。URL、命令、参数、环境变量和密钥继续留在安全边界外。

/rtk 分成 Runtime、Behavior、Session savings 三个板块。ready、unchecked、drifted、unavailable 都有固定图标和完整词。页面还要明说 model projection 只影响模型看到的精简副本，不会改写 Conversation Transcript 或 Session。

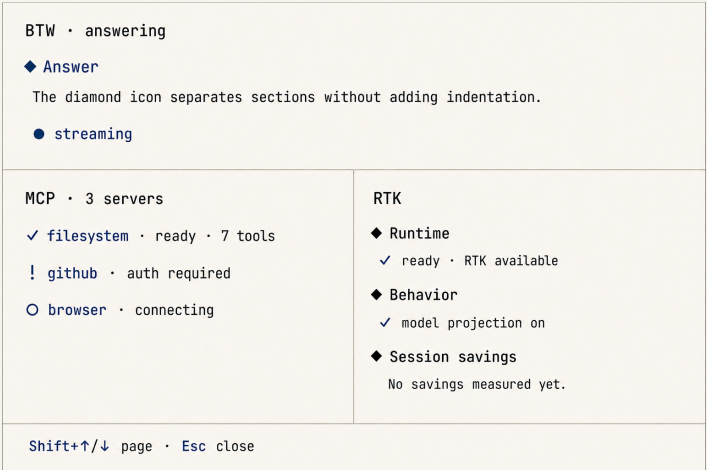


图 4: 浅色目标预览。三个小页面共享板块和状态语言，但没有被强行套进列表详情模板。

/mcp 的具体形状

```
MCP · 2/3 connected · 14 tools
✓ filesystem · connected · 8 tools
! github · needs auth · /mcp-auth github
× browser · failed · /mcp reconnect
browser
■ legacy · disabled
Shift+↑/↓ page · Esc close
```

/rtk 的具体形状

```
RTK · ✓ ready · v0.42.4
◆ Runtime
~/local/bin/rtk · SHA-256 1d8bf5f1...
◆ Behavior
✓ rewriting on · ✓ model projection on
◆ Session savings
12,430 chars (38%) · 24 results
/rtk settings · Esc close
```

边界：/mcp 保持只读，故障转到 /diagnostics，Tool 调用转到 /tools；页面不显示服务器 URL、命令、环境变量、OAuth 数据或 token。/rtk 的示例数字只用来检查排布，Session savings 不是费用承诺；model projection 也不会改写 Transcript、Tool result 或 Session JSONL。

实现顺序与验收方法

设计文档已经完成，但源代码仍是旧页面。实现时应该先落共同语法，再按风险和复用程度逐个改造，最后在宽屏与窄屏上检查真实渲染，不能把这份报告当成完成证明。

第一步实现共享的图标含义、菱形板块标题、Shift+Up/Down 翻页和 Footer 规则。第二步优先改 `/agents`、`/tools`、`/tasks`，因为它们信息量最大，也最能检验列表、详情、实时更新和窄屏退化是否成立。

第三步完成 `/ctx`、`/diagnostics`、`/btw`、`/mcp`、`/rtk`。验收至少覆盖 100×32 与窄宽度，逐项看选择是否稳定、状态是否仍可读、活动是否丢失、Footer 是否挤压正文、Esc 是否按层返回，以及 60% 键盘是否不靠 PageUp/PageDown 也能翻页。

建议的实现批次

批次	内容	为什么先做
1	共享图标、板块标题、翻页别名、Footer 规则	8 个页面都会用到，先定底座能减少返工
2	<code>/agents</code> 、 <code>/tools</code> 、 <code>/tasks</code>	信息最密，包含列表、详情、实时更新和多级返回
3	<code>/ctx</code> 、 <code>/diagnostics</code> 、 <code>/btw</code>	验证操作流程、警告、流式阅读和历史导航
4	<code>/mcp</code> 、 <code>/rtk</code>	静态页风险较低，最后收口状态与低高度适配

逐页验收清单

1. 第一眼：3 秒内能说出页面主角是谁或是什么，不必先读字段名。
2. 选择：> 只表示选中行；双栏只用于反复对照列表与详情，并用栏标题强调色表示焦点；`/agents` 始终单栏。
3. 状态：紧凑列表有图标，详情或静态状态行保留完整状态词。
4. 板块：◆ 只在标题行出现，正文没有额外层层缩进。
5. 翻页：PageUp/PageDown 与 Shift+Up/Down 行为一致，只有溢出时才显示提示。
6. 实时内容：停在底部时自动跟随，向上阅读后不抢滚动位置，并显示有限的新内容提示。
7. 窄屏：先丢描述、计数和次要元数据，不能先丢名称、状态、选中项或 Escape。
8. 返回：Esc 每次只退一层，Raw、详情、列表、Dialog 的顺序稳定。
9. 权威边界：同一份 Agent、Tool、Todo、BTW 或 Permission 状态不在多个常驻位置重复。
10. 空与错：空页面、加载、不可用和错误都有普通话说明，同时保留真实状态与退出路径。

完成标准：只有真实 UI 代码、聚焦测试、100×32 与窄宽度渲染、完整键盘路径和最终 `bun run check` 全部通过，才能把“待实现”改成“已实现”。

附录

A. 本报告依据的决策文件

- docs/research/agent-activity-ui-reference.md
- docs/adr/0010-fold-continuous-retrieval-segments.md
- packages/pi-stuff/src/background-work/README.md
- docs/adr/0008-own-the-context-command-surface.md
- docs/adr/0004-route-suite-diagnostics-through-owned-ui.md
- packages/pi-stuff/src/btw/README.md
- packages/pi-stuff/src/mcp/README.md
- packages/pi-stuff/src/rtk/README.md

B. 术语表

Command Dialog: 用户明确打开的临时全宽页面。它不是浮在 Transcript 上的小窗，也不是长期占位的仪表盘。

Activity: 一段按真实先后顺序发生、对用户有意义的工作记录。它可以包含 Agent 消息、Tool 调用、结果和有限预览。

Capability: Pi Stuff 包里的一个能力模块，例如 Agents、Context 或 Diagnostics。它是问题来源，不一定是用户最先需要看的标题。

Raw protocol: 原始调用数据，给调试用。日常阅读优先看工具已经格式化好的标题、目标、摘要和结果。

Compartment: 从旧对话整理出来的压缩分区。它帮助控制模型上下文大小，不是另一份 Pi 对话。

Historian: 负责创建或重建这些压缩分区的 Context 后台整理器。

model projection: 发给模型看的精简副本。它不会改写用户看到的 Transcript、Tool result 或 Session 文件。

C. 预览图说明

4 张 PNG 均由 ImageGen 生成，使用浅色 Kami 配色。它们用于检查布局、主次、密度与图标位置，不是当前产品截图。图像生成无法可靠承担所有精确字符，因此本报告同时给出等宽文字示例；当图与示例冲突时，以已经写入决策文件的文字为准。

REVIEW STATUS

内容依据已接受决策整理。报告本身完成后仍需单独复核页面密度、字体、图片清晰度和每个原子事实是否保留。